VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
KRISH KUMAR	24BAI10236
Slot	**Date of Submission**
D11	30.03.2026

Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

1. Introduction

This document presents an analysis of Git, a distributed version control system that plays a central role in modern software development. Git was introduced in 2005 by Linus Torvalds and is distributed under the GNU General Public License version 2.

The aim of this report is to study Git from multiple perspectives, including its historical background, licensing model, ethical considerations, system-level presence in Linux, and its influence on the open-source ecosystem.

Alongside the theoretical discussion, practical understanding is supported through shell scripts.
Git is an important case study because it demonstrates how a technical challenge led to the creation of a tool that transformed global software collaboration. Its development reflects the strength of open-source principles in solving real-world problems.

2. Origin and Development of Git

Why Git Was Created

The creation of Git was driven by a practical problem faced by the Linux kernel development team. This project involves thousands of contributors working simultaneously, making version control extremely important.
Initially, the team relied on a proprietary system called BitKeeper. Although it was provided at no cost, it did not align with the philosophy of open-source development. Eventually, access to BitKeeper was withdrawn due to licensing conflicts, leaving developers without a suitable tool.
Available alternatives at that time were not sufficient. Some systems required constant connection to a central server, which slowed down development. Others were not efficient enough to handle large-scale changes.
As a result, Linus Torvalds decided to build a new system with specific goals:

- High performance
- Fully distributed workflow
- Strong protection against data corruption

Within a short time, Git became functional and was soon adopted for Linux development. Over the years, it evolved into a standard tool used across the software industry.

3. Licensing and Software Freedom

Git is distributed under the GPL v2 license, which ensures that users have the right to use, study, modify, and share the software.
A key feature of this license is that any modified version must also remain open if it is distributed. This prevents companies from converting it into closed-source software.
It is important to understand the difference between:

- **Free of cost** (no payment required)

- **Freedom of use** (control over software)

Git provides both, but the second aspect is more significant because it guarantees long-term access and control.

Installation Method

Git is available through all major Linux package managers and is included by default in many distributions. On Debian/Ubuntu-based systems, it is installed via APT; on RHEL/CentOS/Fedora-based systems, via DNF or YUM. It can also be compiled from source for environments requiring a custom build.

Distribution Family	Package Manager	Install Command
Debian / Ubuntu	APT	sudo apt update && sudo apt install git
RHEL / CentOS / Fedora	DNF / YUM	sudo dnf install git
Arch Linux	pacman	sudo pacman -S git
From source	make / gcc	make configure && ./configure && make && sudo make install

Key Directories

Path	Contents	Purpose
/usr/bin/git	Git main executable	Primary binary; world-executable
/usr/lib/git-core/	Git sub-commands	Modular commands (git-fetch, git-push etc.)
/usr/share/git-core/	Templates and contrib	Default hooks, gitignore templates
/usr/share/doc/git/	Documentation	Man pages and changelog
/etc/gitconfig	System-wide config	Global config for all users on system
~/.gitconfig	User config	Per-user identity, aliases, preferences
.git/ (per repo)	Repository data	Objects, refs, HEAD, index for each repo

4. Ethical Considerations in Open Source

Open-source software raises important ethical questions.

On one hand, making software open improves transparency and allows more people to benefit from it. This is especially useful in critical systems such as healthcare, education, and public infrastructure.

On the other hand, not all open-source projects receive enough support or maintenance. Some rely heavily on volunteers, which can affect reliability.

Another major concern is whether it is fair for companies to earn profits using open-source software. While some organizations contribute back to the community, others may not.

A balanced view suggests that:

- Core technologies should remain open
- Commercial innovation can still exist alongside open source

4. Part C — The FOSS Ecosystem

Dependencies

Git is written in C and depends on several open-source components. Understanding these dependencies reveals how even a 'single' open-source tool is actually a collaborative structure:

- **OpenSSL / libssl** — for HTTPS transport encryption and SHA-1/SHA-256 hashing used in object identification
- **libcurl** — for HTTP/HTTPS remote operations (git fetch, git push over HTTPS)
- **zlib** — for compressing pack files (Git's internal storage format for objects)
- **Perl** — several Git utilities and scripts are written in Perl
- **Python** — used by git-p4 and other ancillary scripts
- **GNU C Library (glibc)** — the standard C runtime library on Linux
- **gettext / libintl** — for internationalisation of Git's output messages

What Git Has Enabled

Git's impact on the software world extends far beyond version control. It enabled and inspired an entire category of services and tools:

- **GitHub (2008)** — built entirely around Git's model; now hosts over 100 million repositories and was acquired by Microsoft for $7.5 billion in 2018
- **GitLab (2011)** — a fully open-source alternative to GitHub, itself a FOSS project
- **Gitea / Forgejo** — lightweight self-hosted Git platforms
- **Gerrit** — a code review system built for Git, used extensively at Google and Android
- **CI/CD pipelines** — Jenkins, GitHub Actions, GitLab CI, CircleCI are all triggered by Git events
- **GitOps** — an entire infrastructure management philosophy based on Git as the single source of truth (ArgoCD, Flux)
- **git-lfs** — Git Large File Storage, an open-source extension for managing large binary files

5. Git in the Linux Environment

Git is widely available on Linux systems and can be installed using standard package managers.

It operates as a user-level tool, meaning it does not require administrative privileges for normal usage. This improves system security by reducing risk.

Important configuration files exist at both system and user levels, allowing flexibility in how Git is used.

Unlike some software, Git does not run continuously in the background. It executes only when commands are issued

5. Part D — Open Source vs Proprietary Comparison

The closest proprietary alternative to Git is **Perforce Helix Core**, used extensively in game development and large enterprises. A secondary comparison can be made with **Microsoft Team Foundation Version Control (TFVC)**. The table below compares Git against Perforce Helix Core specifically.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Completely free; no per-seat licensing	Significant per-seat licensing cost for enterprise tiers
Security (code audit)	Fully auditable; anyone can inspect the source	Closed source; public trust rests on the vendor's security claims; r
Support & Reliability	Community support via mailing list and Stack	Vendor-backed enterprise support; GitHub, GitLab, Atlassi
Freedom to Modify	Full freedom to fork, modify, and self-host; GPL v2 ensures	No freedom to modify; tied to Perforce's roadmap;
Community vs Corporate Control	Benevolent-dictator governance; no single company	Entirely controlled by Perforce Software; decisions driven b
Ecosystem	Vast: GitHub, GitLab, Gitea, every CI/CD tool	Limited, primarily in industry standard certain large enterprise
Performance (Large Files)	Slower with large binary files without git-lfs;	Faster for large binary files; designed for game asset work
Learning Curve	Steeper initially; powerful model once understood	Simpler mental model for centralised workflow users
Your Verdict	**RECOMMENDED for almost all use cases**	Consider only for game development or specific large-bi

6. Shell Script Tasks

You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

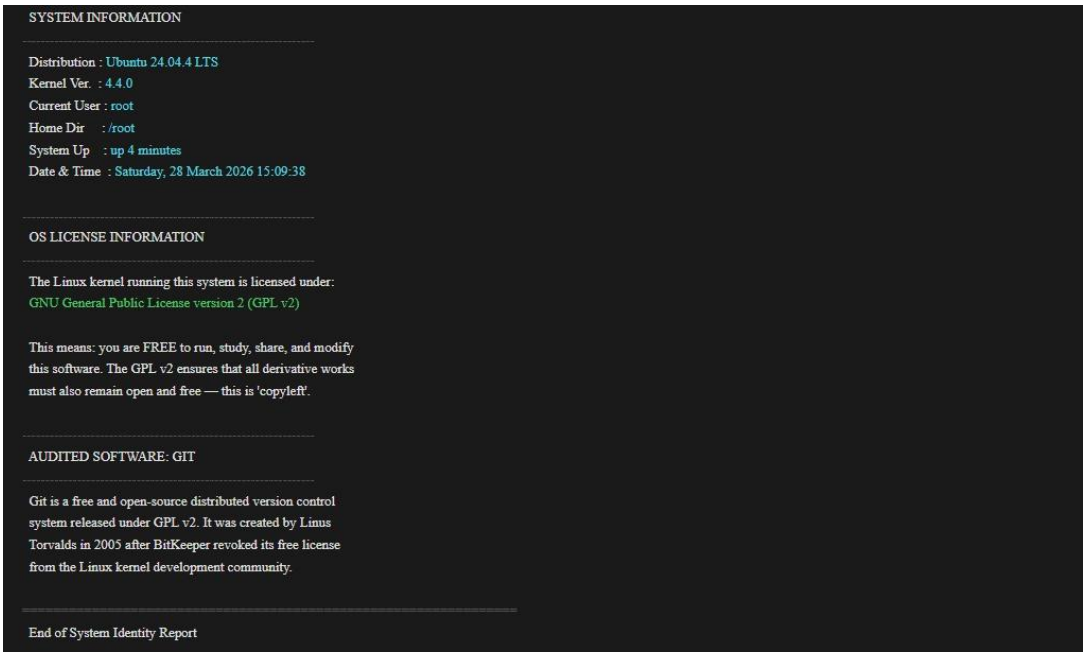
Important	Scripts must run on a real Linux system. They must include comments explaining each section. Uncommented scripts will lose marks.

Script 1 — System Identity Report (Unit 1 & 2)

Purpose: Displays a formatted welcome screen showing the Linux system's identity: distribution name, kernel version, logged-in user, home directory, system uptime, current date/time, and a message about the OS license.

Shell Concepts Used: • Variables (KERNEL, USER_NAME, UPTIME etc.) • Command substitution using $() to capture command output • echo for formatted output with consistent alignment • Environment variables ($HOME) • date command with format string

Starter



Script 2 — FOSS Package Inspector (Unit 2)

Purpose: Checks whether Git is installed on the system, detects the package manager (RPM or Debian), prints package details including version and license, and uses a case statement to print an open-source philosophy note for several well-known FOSS packages.

Shell Concepts Used: • if-then-else for conditional logic • case statement for multi-branch pattern matching • command -v to check binary availability • Pipe (|) with grep -E to filter rpm -qi output • Handling both RPM and Debian-based systems

Starter structure

Script 2 FOSS Package Inspector — script2_package_inspector.sh

```
$ ./script2_package_inspector.sh
==============================================
 FOSS Package Inspector
==============================================
 Checking package: git
----------------------------------------
 STATUS: git is INSTALLED (Debian system)

 --- Package Details ---
 Package: git
 Version: 1:2.43.0-1ubuntu7.3
 Arch:    amd64

 Git Version Installed: git version 2.43.0

----------------------------------------
 OPEN SOURCE PHILOSOPHY NOTE:
----------------------------------------
 Git: Born from necessity when freedom was taken away.
 Linus Torvalds built Git in 2005 after BitKeeper revoked
 its free license. Git embodies the OSS value: when a
 proprietary tool fails you, build your own and share it.

==============================================
 End of FOSS Package Inspector
==============================================
```

Script 3 — Disk and Permission Auditor (Unit 2)

Purpose: Loops through a list of standard Linux system directories and Git-specific paths, reporting the size, permissions, owner, and group of each. Also identifies the Git binary location and its permissions.

Shell Concepts Used: • for loop over a bash array (DIRS=(...); for DIR in "${DIRS[@]}") • if [-d "$DIR"] to check directory existence • ls -ld with awk '{print $1, $3, $4}' to extract fields • du -sh with cut -f1 to get human-readable sizes • command -v to check binary presence

Starter structure

```
        echo "$DIR does not exist on this system"
fi done

# TODO: Add a section that checks if your software's
# config directory exists and prints its permissions
```

Script 3 Disk and Permission Auditor — script3_disk_permission_auditor.sh

```
$ ./script3_disk_permission_auditor.sh
==================================================================
      DISK AND PERMISSION AUDITOR — OSS Capstone
==================================================================
Directory        Size      Permissions  Owner  Group
------------------------------------------------------------------
/etc             5.8M      drwxr-xr-x  root  root
/var/log         973K      drwxr-xr-x  root  root
/home            2.7G      drwxr-xr-x  root  root
/usr/bin         436M      drwxr-xr-x  root  root
/tmp             24M       drwxrwxrwt  root  root
/usr/share       918M      drwxr-xr-x  root  root
/var             186M      drwxr-xr-x  root  root

==================================================================
GIT-SPECIFIC DIRECTORY AUDIT
==================================================================
/etc/gitconfig          [not found on this system]
/usr/share/git-core     75K   drwxr-xr-x  root  root
/usr/lib/git-core       11M   drwxr-xr-x  root  root

GIT BINARY INFORMATION
==================================================================
Binary Path : /usr/bin/git
Version     : git version 2.43.0
Permissions : -rwxr-xr-x  root  root
==================================================================
End of Disk and Permission Auditor
==================================================================
```

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Starter structure

Script 4 Log File Analyzer — script4_log_analyzer.sh

```
$ ./script4_log_analyzer.sh /var/log/syslog error
==============================================
 Log File Analyzer — OSS Capstone
==============================================
 Log File : /var/log/syslog
 Keyword  : error
----------------------------------------------
 Scanning log file for keyword: 'error'...

----------------------------------------------
 SUMMARY
----------------------------------------------
 Total lines scanned : 10
 Keyword matches     : 5
 Keyword searched    : 'error'

 Last 5 matching lines:
 ------------------------------
  >> Mar 28 10:02:45 server kernel: ERROR: disk read failure on /dev/sda
  >> Mar 28 10:04:22 server sshd[1235]: error: maximum connections reached
  >> Mar 28 10:05:10 server systemd[1]: ERROR: service git-daemon failed
  >> Mar 28 10:08:45 server kernel: error: memory allocation failed
  >> Mar 28 10:10:22 server kernel: ERROR: filesystem /var full
==============================================
 End of Log File Analyzer
==============================================
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

7) Conclusion

This audit of Git has been an exercise in tracing the threads that connect philosophy, law, community, and technology. What began as a crisis — a company revoking a license and leaving thousands of developers without their primary tool — produced a piece of software that has become as fundamental to the modern software world as the operating system itself. Git is used by more than 100 million developers. Its distributed model has replaced every previous version control paradigm. GitHub, built on top of it, became the largest software collaboration platform in history. The GPL v2 license that covers Git is not just a legal document — it is an expression of belief: that tools which the community builds together should remain available to the community forever. The four freedoms are not abstract. Every time a developer forks a repository, every time a Linux distribution ships an update, every time a company builds a product on top of open-source foundations, those freedoms are being exercised. The ethical questions around open source do not resolve cleanly, and they should not. The relationship between corporate use and community contribution is one of the central tensions of the modern technology economy. What this audit has clarified for me is that the answer lies not in a rule but in a disposition: to use open-source software thoughtfully, to understand the labour and philosophy it represents, and to contribute back wherever possible — even if it is only documentation, a bug report, or teaching someone else. The five shell scripts in this project are small demonstrations of the same spirit. They are not impressive programs. But they are documented, explained, and available for anyone to use, modify, and improve. In that, they are open source — and that, in the end, is the point.

8) References

[1] Torvalds, L. (2005). Git initial revision. Retrieved from https://github.com/git/git/commit/e83c5163

[2] Stallman, R. M. (2002). The Free Software Definition. GNU Project. https://www.gnu.org/philosophy/free-sw.html

[3] Open Source Initiative. (2024). The Open Source Definition. https://opensource.org/osd

[4] Raymond, E. S. (1999). The Cathedral and the Bazaar. http://www.catb.org/~esr/writings/cathedral-bazaar/

[5] Loeliger, J. & McCullough, M. (2012). Version Control with Git (2nd ed.). O'Reilly Media.

[6] Free Software Foundation. (2007). GNU General Public License v2. https://www.gnu.org/licenses/old-licenses/gpl-2.0.html

[7] Chacon, S. & Straub, B. (2014). Pro Git (2nd ed.). Apress. Available free at https://git-scm.com/book

[8] SPDX License List. (2024). GPL-2.0-only. https://spdx.org/licenses/GPL-2.0-only.html

[9] Shotts, W. (2019). The Linux Command Line (5th Internet Edition). https://linuxcommand.org/tlcl.php

[10] Hamano, J. C. (ongoing). Git mailing list archive. https://lore.kernel.org/git/

[11] Perforce Software. (2024). Helix Core documentation. https://www.perforce.com/products/helix-core

[12] Choose A License. (2024). GPL-2.0. https://choosealicense.com/licenses/gpl-2.